深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

SUAT

数字逻辑 Digital Logic

课程编码： FCAA213 | 开课单位：计算机科学与人工智能学院

开课时间：2026-2027学年秋季学期

主讲教师：马懋德教授 与 李嘉龙助理教授



深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

目录 Contents

Chapter 1 Introduction
Chapter 2 Number & Arithmetic
Chapter 3 Logic Functions
Chapter 4 Verilog Language
Chapter 5 Combinational Circuits
Chapter 6 Basic Storage Elements
Chapter 7 Synchronous Sequential Circuits
Chapter 8 Asynchronous Sequential Circuits
Chapter 9 Digital System Design

深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

Chapter 7 Synchronous Sequential Circuits

7.1 Finite State Machine

7.2 Design of A Smiling Snail

7.3 Design of A Register Swap

7.4 Design of A Counter

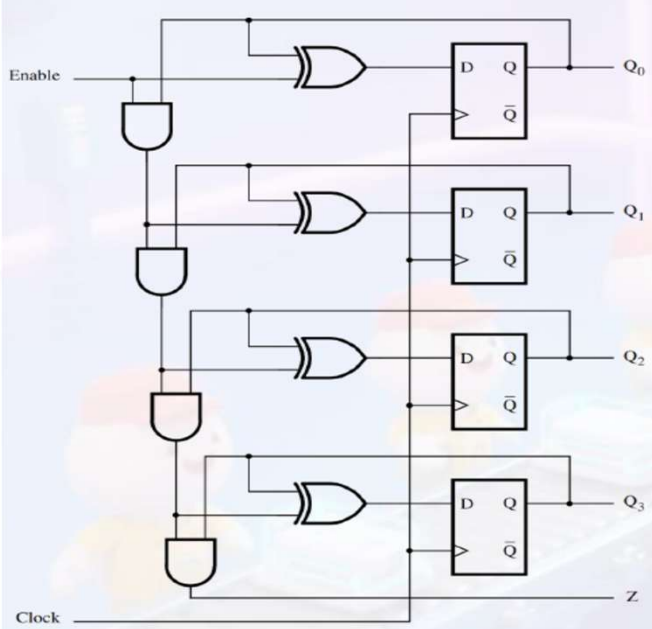
7.5 Mealy-type FSM

7.6 Analysis of Synchronous Sequential Circuits

同步时序电路

Sequential Circuit (时序电路)

- The general form of a sequential circuit



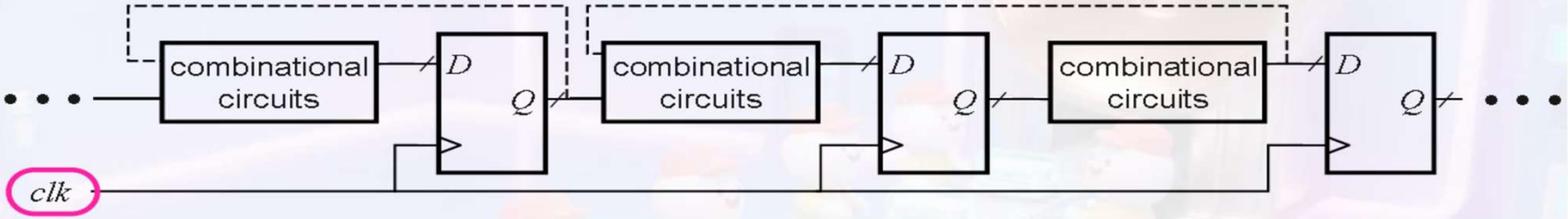


- Feedback paths exist in sequential circuits
- The outputs depend on the past behavior of the circuit

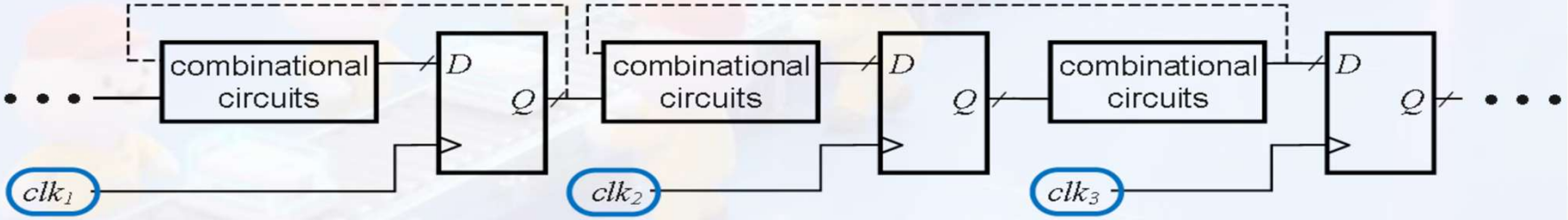
同步时序电路

Synchronous vs. Asynchronous

- Synchronous sequential circuit 同步时序电路



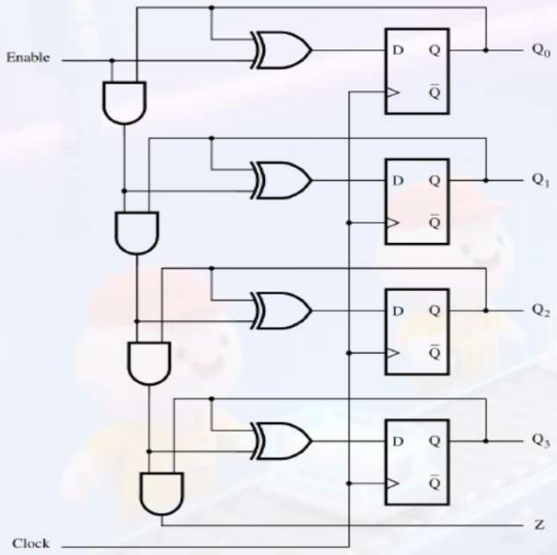
- Asynchronous sequential circuit 异步时序电路



同步时序电路

Finite State Machine (FSM 有限状态机)

- Sequential circuits are also called finite state machines (FSMs)
- Why the name: the **functional behavior** of sequential circuits can be represented using a finite number of states.



Decimal	Binary			
	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

How many states are in this sequential circuit (counter)?

同步时序电路

A Design Example of FSM: smiling snail



A robotic snail crawls along a paper tape containing a sequence of 1's and 0's. On each clock cycle, it crawls to the next bit.

Assume the snail will smile when the last two bits that it has crawled over are '11'. Design a circuit for the snail to compute when it should smile.

蜗牛爬过两个连续的"1"，就微笑。 Easy for human !

同步时序电路

Smiling Snail

- Circuit specification
  - w : the input digit
  - z : output action (smile "1", no smile "0")
  - all changes occur on the positive edge of the clock signal
  - $z = 1$, if "$w = 1$" happens during two preceding clock cycles
    $z = 0$, otherwise

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

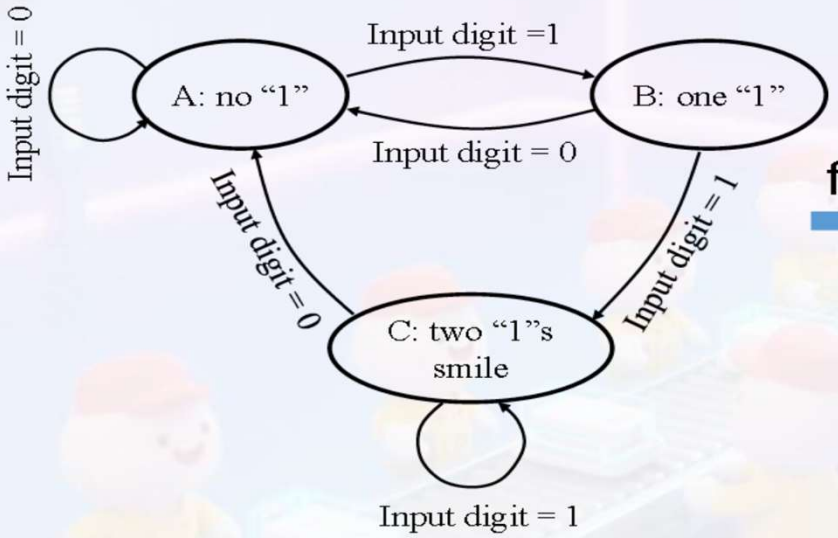



同步时序电路

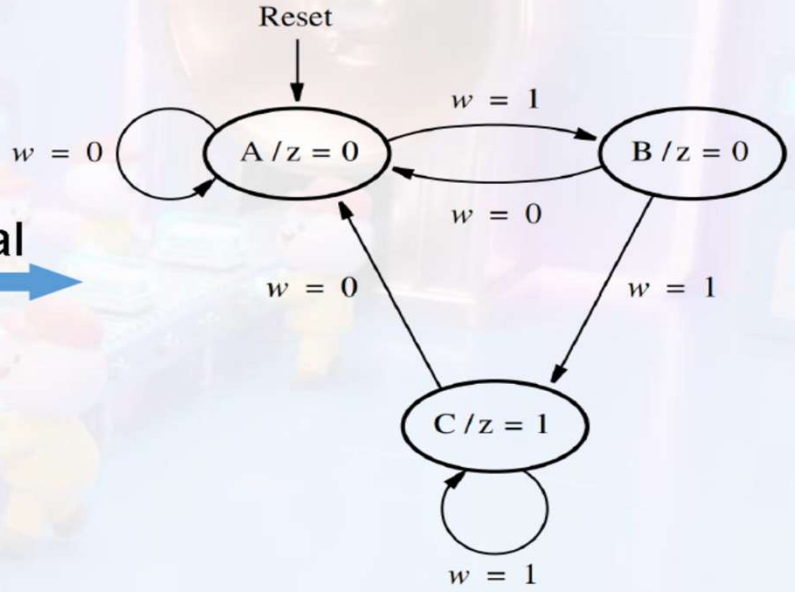
Smiling Snail

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

- Need consecutive two "1" to smile



formal

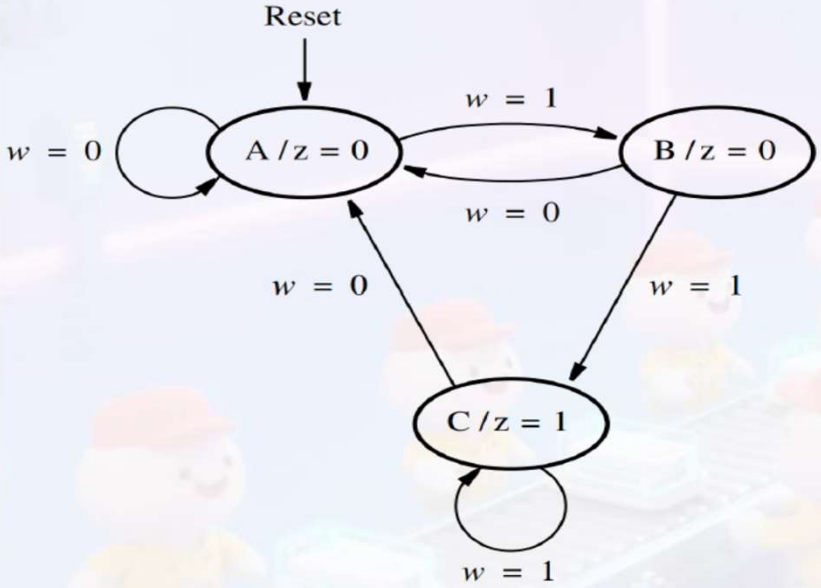


State diagram 状态图

同步时序电路

Smiling Snail

- State diagram & state table



State diagram 状态图

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

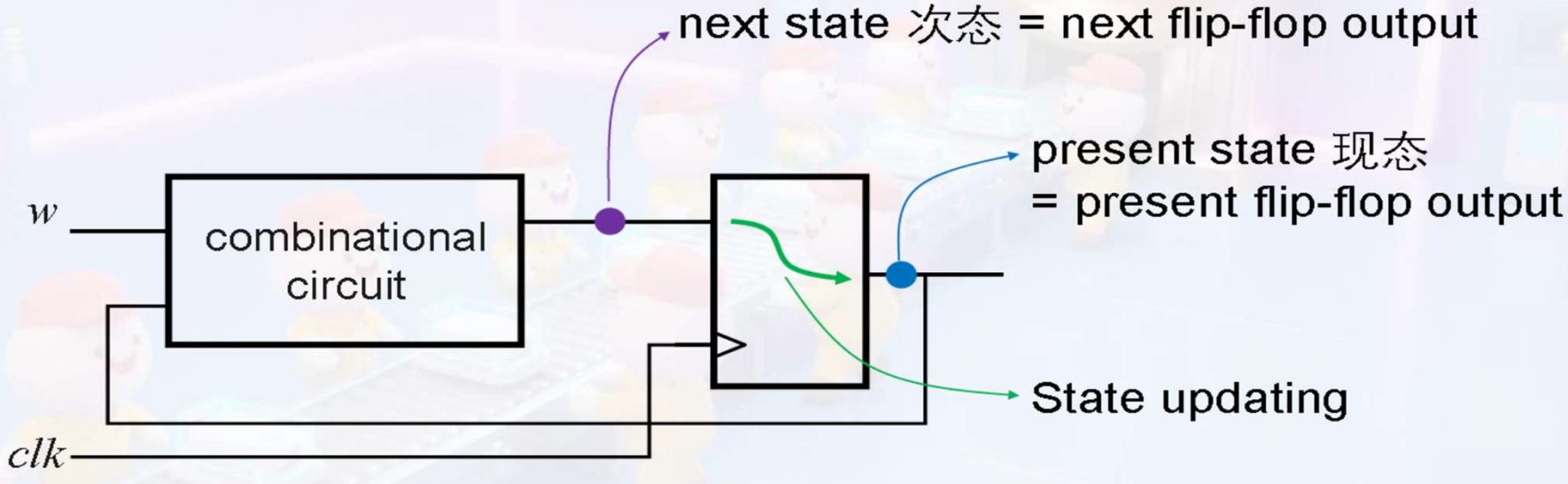
Assume that the first state in the table is the starting state

State table 状态表

同步时序电路

Smiling Snail

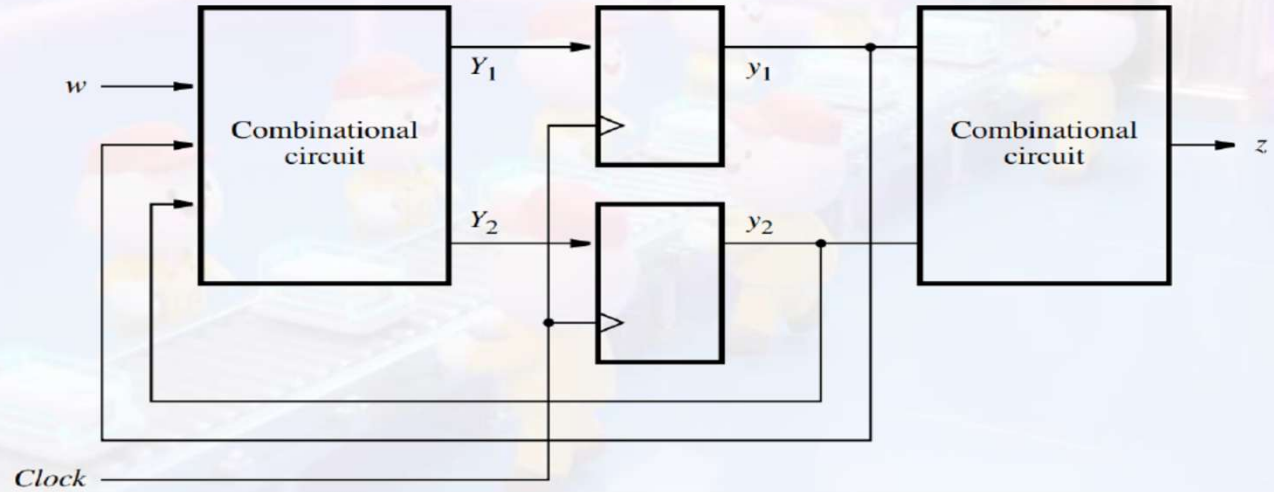
- States can be indicated by flip-flop outputs
- State updating can be implemented by flip-flop updating



同步时序电路

Smiling Snail

- 1 flip-flop can denote 2 states, 3 states (A, B, C) need 2 flip-flops
- State variable 状态变量
  - y_1, y_2 : present-state variable 现态变量
  - Y_1, Y_2 : next-state variable 次态变量



同步时序电路

Smiling Snail

- State assignment 状态分配
  - Assign specific valuation of variables to each state

Present state	Next state $w=0$	Next state $w=1$	Output z
A	A	B	0
B	A	C	0
C	A	C	1



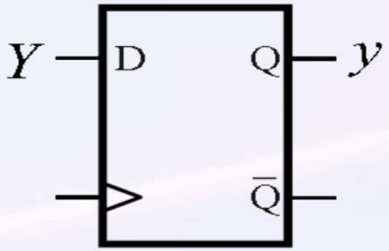
	Present state $y_2 y_1$	Next state $w=0$ $Y_2 Y_1$	Next state $w=1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

State-assigned table 状态分配表

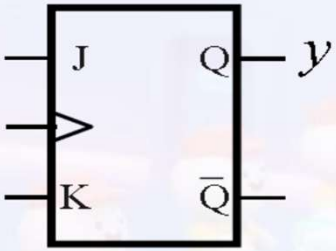
同步时序电路

Smiling Snail

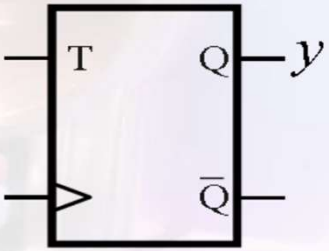
- Choice of flip-flops 确定触发器类型



y	Y	D
0	0	0
0	1	1
1	0	0
1	1	1



y	Y	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	1	0



y	Y	T
0	0	0
0	1	1
1	0	1
1	1	0

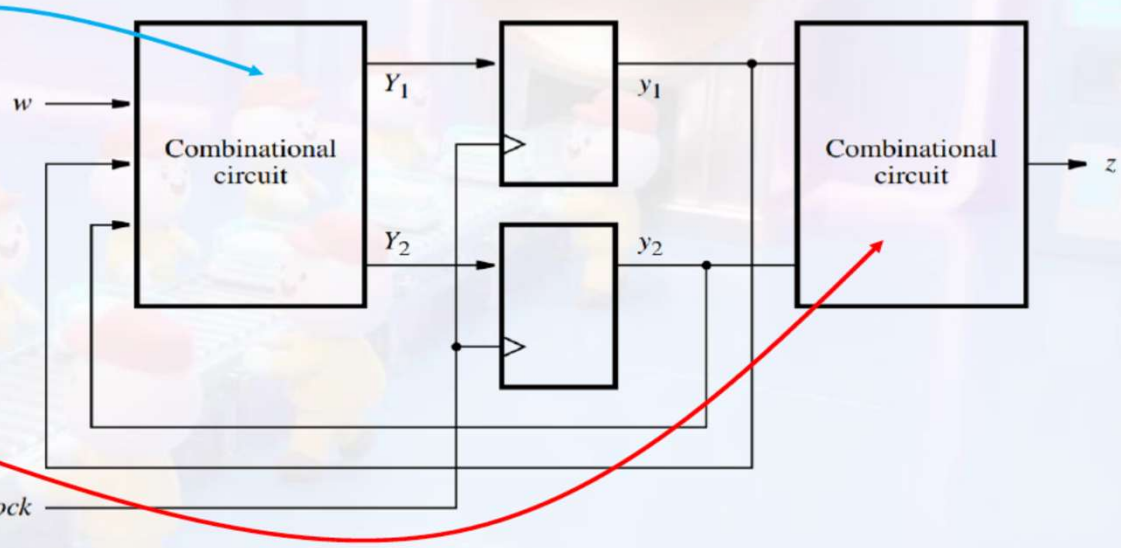
$D = Y$ The most straightforward choice: D flip-flop

同步时序电路

Smiling Snail

- Derivation of Next-State and Output Expressions

	Present state $y_2 y_1$	Next state $w = 0$ $Y_2 Y_1$	Next state $w = 1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*



同步时序电路

Smiling Snail

- Derivation of Next-State and Output Expressions
  - Convert state-assigned table to truth table, if necessary

	Present state	Next state $w = 0$	Next state $w = 1$	Output
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

w	y_2	y_1	Y_2	Y_1
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	d	d
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	d	d

y_2	y_1	z
0	0	0
0	1	0
1	0	1
1	1	d

同步时序电路

Smiling Snail

- Derivation of Next-State and Output Expressions

for Y_2

w \ y_2y_1	00	01	11	10
0			d	
1		1	d	1

$$Y_2 = wy_1 + wy_2 = w(y_1 + y_2)$$

for Y_1

w \ y_2y_1	00	01	11	10
0			d	
1	1		d	

$$Y_1 = w\bar{y}_1\bar{y}_2$$

for z

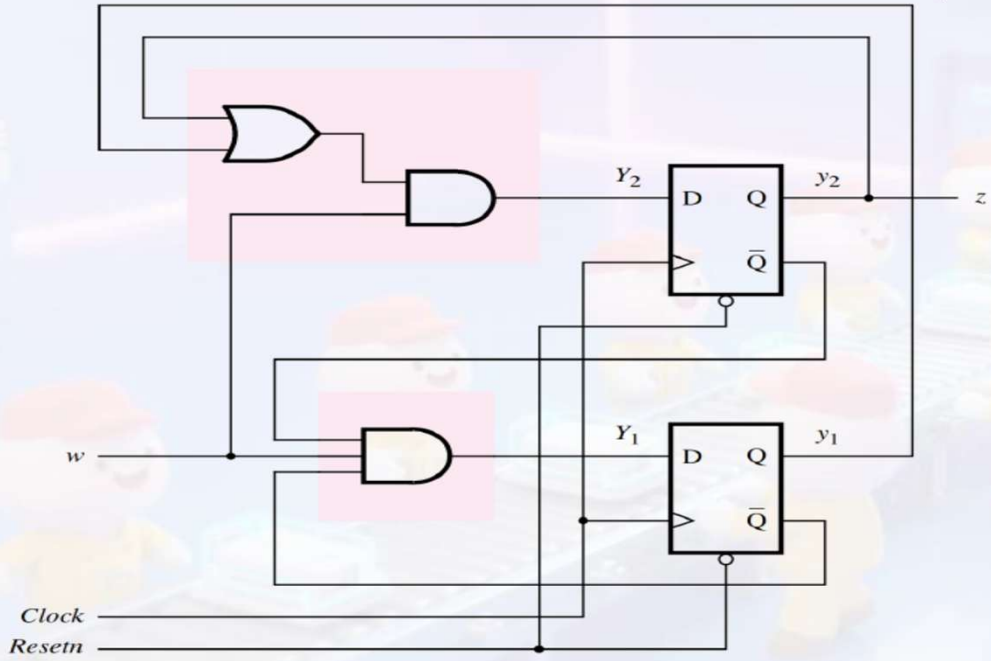
y_2 \ y_1	0	1
0		
1	1	d

$$z = y_2$$

同步时序电路

Smiling Snail

- Derivation of Next-State and Output Expressions



$$Y_2 = wy_1 + wy_2 = w(y_1 + y_2)$$

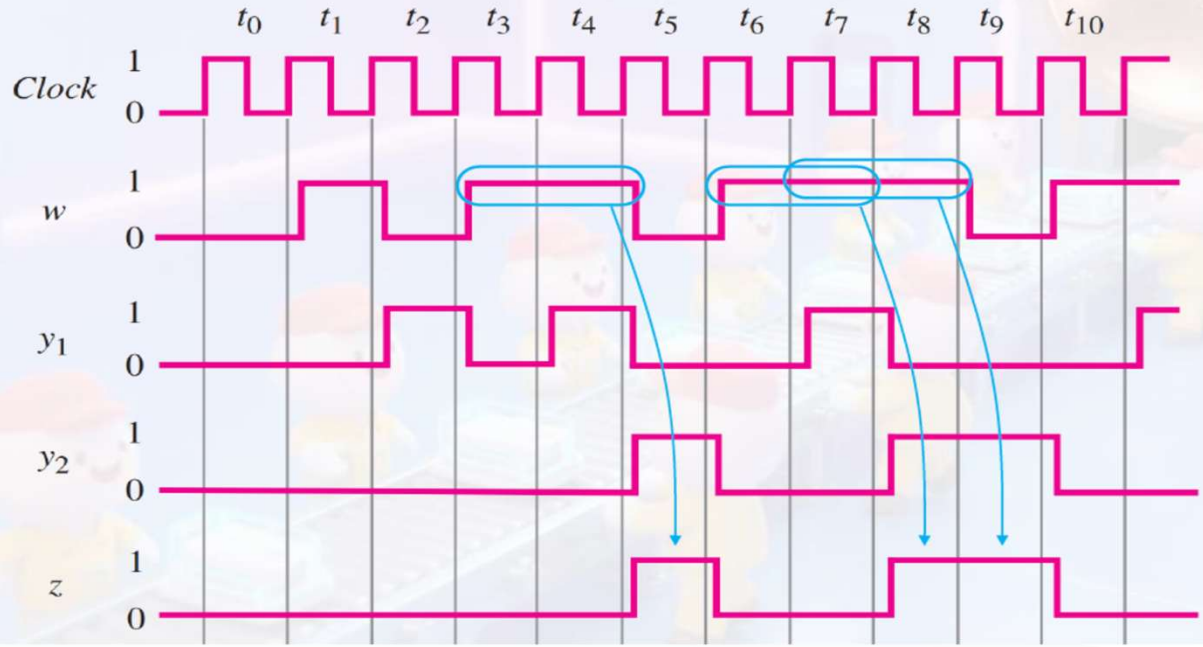
$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$z = y_2$$

同步时序电路

Smiling Snail

- Derivation of Next-State and Output Expressions





$$Y_2 = w(y_1 + y_2)$$

$$Y_1 = w\bar{y}_1\bar{y}_2$$

$$z = y_2$$

同步时序电路

Practical Application of the Circuit

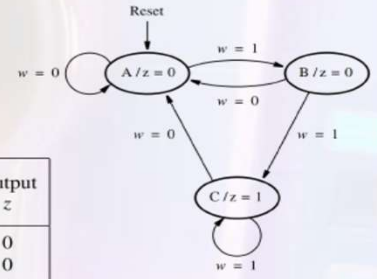
- Automatically-controlled Vehicle
  - The vehicle is designed to run at some predetermined speed. If the speed exceeds the desirable limit, it has to be slowed down
  - The speed is measured at regular intervals
  - If speeding happens during two or more consecutive measurements, a control signal will be asserted to slow the vehicle down 连续监测到两次超速则刹车



同步时序电路

Summary of Design Steps

1. Obtain the circuit specification → Input, output, function
2. Derive the state diagram



3. Create state table from the state diagram

Present state	Next state $w=0$	Next state $w=1$	Output z
A	A	B	0
B	A	C	0
C	A	C	1

4. Determine the number of state variables and perform the state assignment

	Present state y_2y_1	Next state $w=0$ Y_2Y_1	Next state $w=1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

5. Choose the type of flip-flops and derive logic expressions of next-state and output

w \ y_2y_1	00	01	11	10
0			d	
1		1	d	1

$Y_2 = w(y_1 + y_2)$

6. Implement the circuit as indicated by the logic expressions

同步时序电路

Practice:

Problem: Derive the state table (状态表) for the following FSM.

A robotic snail crawls along a paper tape containing a sequence of 1's and 0's. On each clock cycle, it crawls to the next bit.

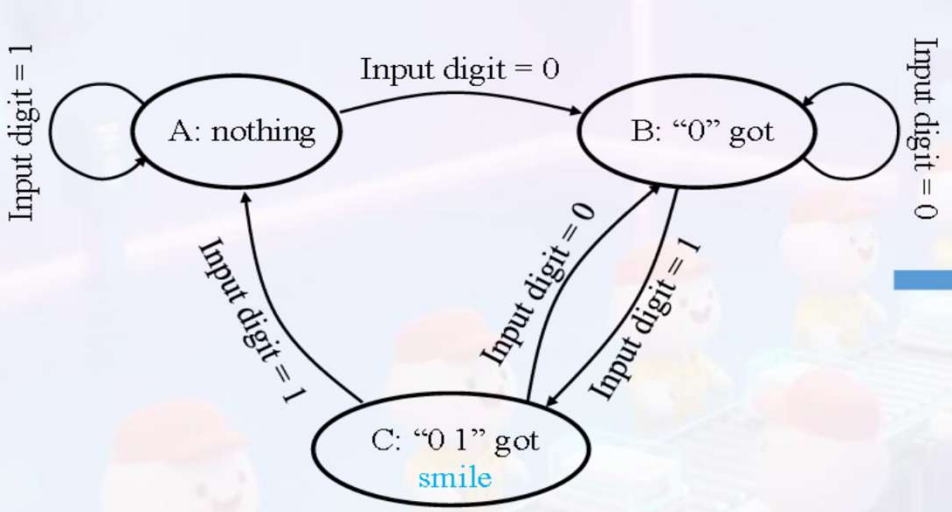
Assume the snail will smile when the last two bits that it has crawled over are '**0, 1**'. Design a circuit for the snail to compute when it should smile.

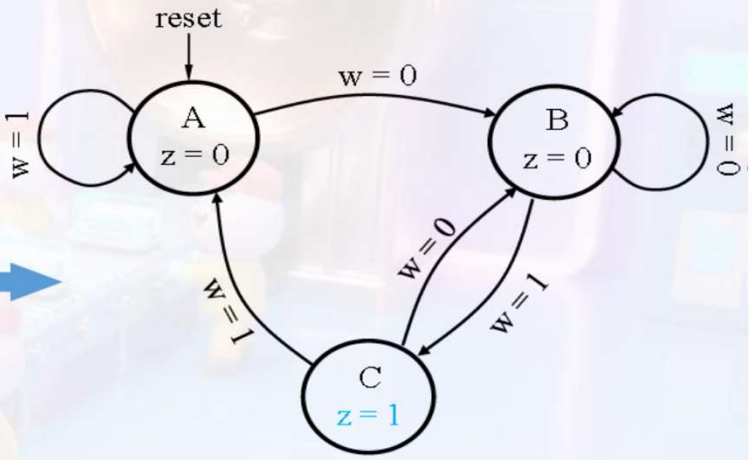
蜗牛连续爬过"**0, 1**"，就微笑。

同步时序电路

Practice:

- Need “0” and then “1”

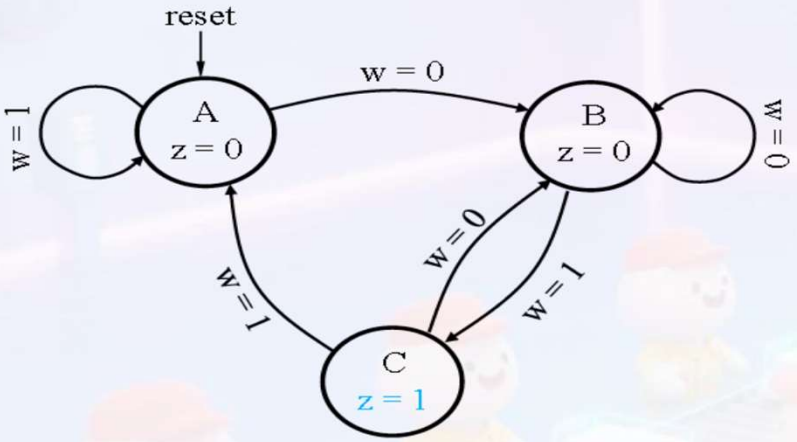




同步时序电路

Practice:

- State diagram & state table



Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	B	A	0
B	B	C	0
C	B	A	1

Assume that the first state in the table is the starting state

State diagram 状态图

State table 状态表

同步时序电路

State-Assignment Problem 状态分配问题

- **Issue**: is the design the optimal one?
- Different state diagrams, state assignments lead to different designs

- Example: a possible alternative state assignment

	Present state $y_2 y_1$	Next state $w = 0$ $Y_2 Y_1$	Next state $w = 1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

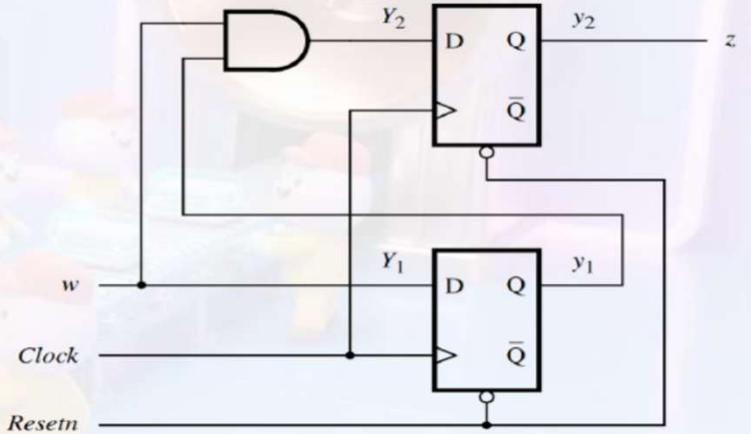
	Present state $y_2 y_1$	Next state $w = 0$ $Y_2 Y_1$	Next state $w = 1$ $Y_2 Y_1$	Output z
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

同步时序电路

State-Assignment Problem

- Through previous procedure, a new circuit design will be obtained

	Present state	Next state		Output
		$w = 0$	$w = 1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	z
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d



$$Y_1 = w$$
$$Y_2 = wy_1$$
$$z = y_2$$

A better design !

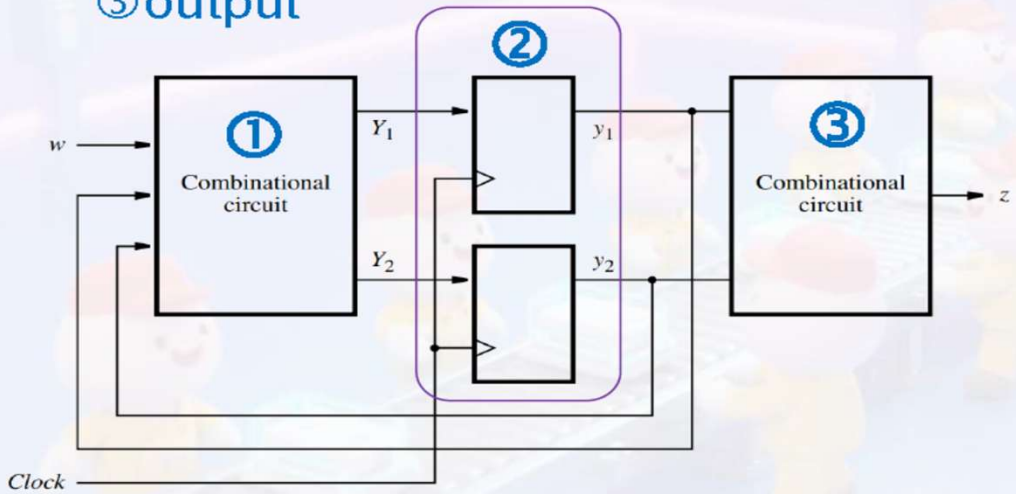
State-Assignment Problem

- For large circuits, different state assignments can have a substantial effect on the cost of the final implementation
- However it is often impossible to find the best state assignment for a large circuit
- CAD tools usually perform the state assignment using heuristic techniques. These techniques are usually proprietary, and details are seldom published

同步时序电路

Verilog code for FSM

- Three major blocks:
  - ①next state logic
  - ②sequential block
  - ③output




```
module simple (Clock, Resetn, w, z);
    input Clock, Resetn, w;
    output z;
    reg [2:1] y, Y;
    parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10;

①   // Define the next state combinational circuit
    always @(w, y)
        case (y)
            A: if (w)       Y = B;
               else         Y = A;
            B: if (w)       Y = C;
               else         Y = A;
            C: if (w)       Y = C;
               else         Y = A;
            default:        Y = 2'bxx;
        endcase

②   // Define the sequential block
    always @(negedge Resetn, posedge Clock)
        if (Resetn == 0)   y <= A;
        else   y <= Y;

③   // Define output
    assign z = (y == C);

endmodule
```

同步时序电路

Verilog code for FSM

- Alternative style of codes: sometimes blocks can be merged

①&③ ②

```
module simple (Clock, Resetn, w, z);
    input Clock, Resetn, w;
    output reg z;
    reg [2:1] y, Y;
    parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10;
    // Define the next state and output combinational circuits
    always @(w, y)
    begin
        case (y)
            A: if (w)   Y = B;
               else     Y = A;
            B: if (w)   Y = C;
               else     Y = A;
            C: if (w)   Y = C;
               else     Y = A;
            default:    Y = 2'bxx;
        endcase
        z = (y == C); // Define output
    end
    // Define the sequential block
    always @(negedge Resetn, posedge Clock)
        if (Resetn == 0) y <= A;
        else y <= Y;
endmodule
```

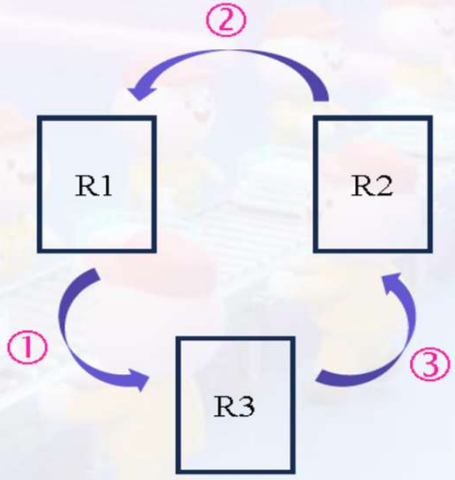
①&② ③

```
module simple (Clock, Resetn, w, z);
    input Clock, Resetn, w;
    output z;
    reg [2:1] y;
    parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10;
    // Define the sequential block
    always @(negedge Resetn, posedge Clock)
        if (Resetn == 0)  y <= A;
        else
            case (y)
                A: if (w)   y <= B;
                   else     y <= A;
                B: if (w)   y <= C;
                   else     y <= A;
                C: if (w)   y <= C;
                   else     y <= A;
                default:    y <= 2'bxx;
            endcase
    // Define output
    assign z = (y == C);
endmodule
```

同步时序电路

Register Swap

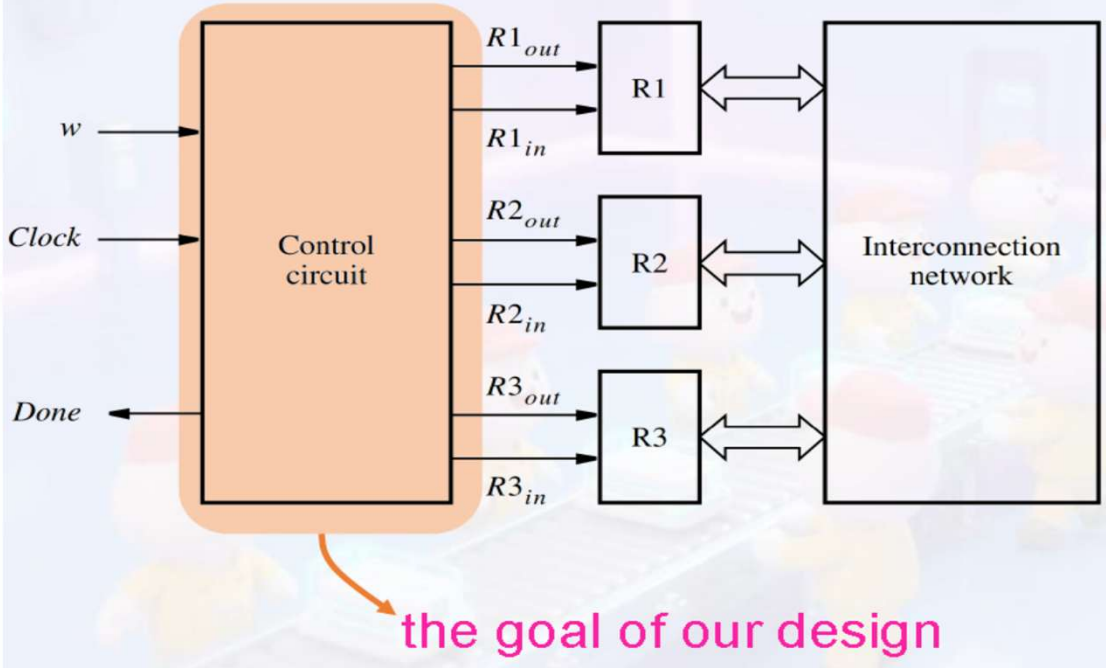
- A computer system usually contains a number of registers that hold data during various operations. Sometimes it is necessary to swap the contents of two registers.



同步时序电路

Register Swap

- A system for register swap

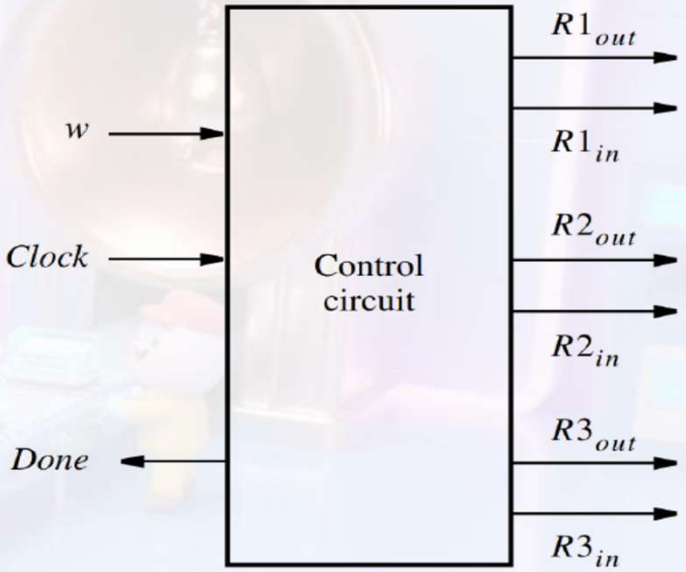


- Registers are connected via an interconnection network
- Each register has two control signals:
  - Rk_{out} : causes data from register Rk into the interconnection network.
  - Rk_{in} : causes data from the network into Rk
- The Rk_{out} and Rk_{in} signals are generated by a control circuit, which is a finite state machine

同步时序电路

Register Swap

- Circuit function
  - Assume the swapping starts when the input w has a duration of one clock cycle
  - The swapping between $R1$ and $R2$:
    1. $R2 \rightarrow R3$, using $R2out = 1$ and $R3in = 1$
    2. $R1 \rightarrow R2$, using $R1out = 1$ and $R2in = 1$
    3. $R3 \rightarrow R1$, using $R3out = 1$ and $R1in = 1$
    4. When the swapping is done, $Done = 1$



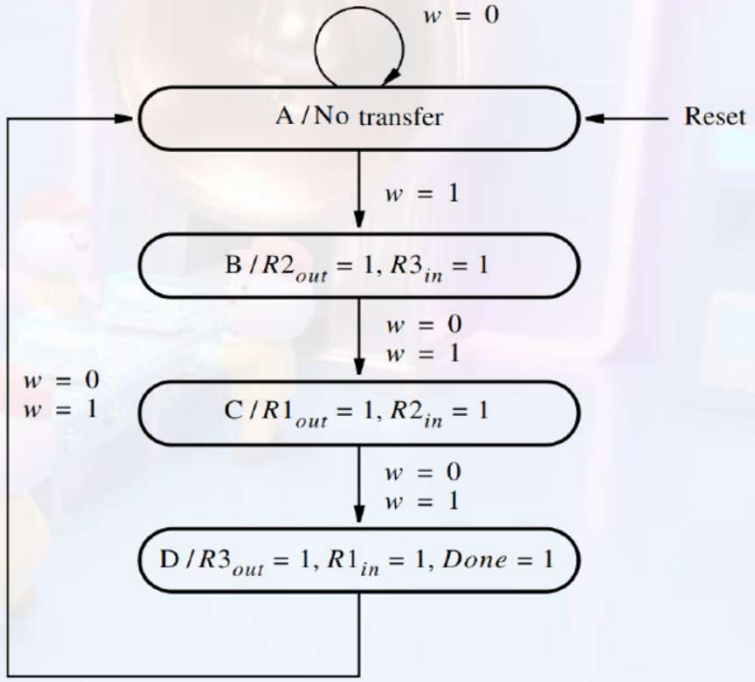
- Circuit specification
  - w : the input digit
  - $R1_{out}$, $R2_{out}$, $R3_{out}$: output control signal for data out of $R1, R2, R3$
  - $R1_{in}$, $R2_{in}$, $R3_{in}$: output control signal for data into $R1, R2, R3$
  - **Done** : output sign signal when the swapping is completed

同步时序电路

Register Swap

- State diagram

- Assume the swapping starts when the input w has a duration of one clock cycle → A
- The swapping between $R1$ and $R2$:
  1. $R2 \rightarrow R3$, using $R2out = 1$ and $R3in = 1$ → B
  2. $R1 \rightarrow R2$, using $R1out = 1$ and $R2in = 1$ → C
  3. $R3 \rightarrow R1$, using $R3out = 1$ and $R1in = 1$ → D
  4. When the swapping is done, $Done = 1$ → D



Register Swap

- State table

Present state	Next state		Outputs						
	$w = 0$	$w = 1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	A	B	0	0	0	0	0	0	0
B	C	C	0	0	1	0	0	1	0
C	D	D	1	0	0	1	0	0	0
D	A	A	0	1	0	0	1	0	1

- State assignment

	Present state	Next state		Outputs						
		$w = 0$	$w = 1$							
	y_2y_1	Y_2Y_1	Y_2Y_1	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 0	1 0	0	0	1	0	0	1	0
C	1 0	1 1	1 1	1	0	0	1	0	0	0
D	1 1	0 0	0 0	0	1	0	0	1	0	1

同步时序电路

Register Swap

- Choice of flip-flops: use **two** D flip-flops
- Derivation of Next-State and Output Expressions

	Present state	Next state		Outputs						
		$w = 0$	$w = 1$							
	y_2y_1	Y_2Y_1	Y_2Y_1	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 0	1 0	0	0	1	0	0	1	0
C	1 0	1 1	1 1	1	0	0	1	0	0	0
D	1 1	0 0	0 0	0	1	0	0	1	0	1

$$Y_1 = w\bar{y}_1 + \bar{y}_1 y_2$$

$$Y_2 = y_1\bar{y}_2 + \bar{y}_1 y_2$$

w	y_2	y_1	Y_2	Y_1
0	0	0	0	0
0	0	1	1	0
0	1	0	1	1
0	1	1	0	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	1
1	1	1	d	d

同步时序电路

Register Swap

- Choice of flip-flops: use **two** D flip-flops
- Derivation of Next-State and Output Expressions

	Present state	Outputs						
	$y_2 y_1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0	0	0	0	0	0	0
B	0 1	0	0	1	0	0	1	0
C	1 0	1	0	0	1	0	0	0
D	1 1	0	1	0	0	1	0	1

y_2	y_1	$R1_{out}$ $R2_{in}$	$R2_{out}$ $R3_{in}$	$R3_{out}$ $R1_{in}$ $Done$
0	0	0	0	0
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

$$R1_{out} = R2_{in} = \bar{y}_1 y_2$$

$$R1_{in} = R3_{out} = Done = y_1 y_2$$

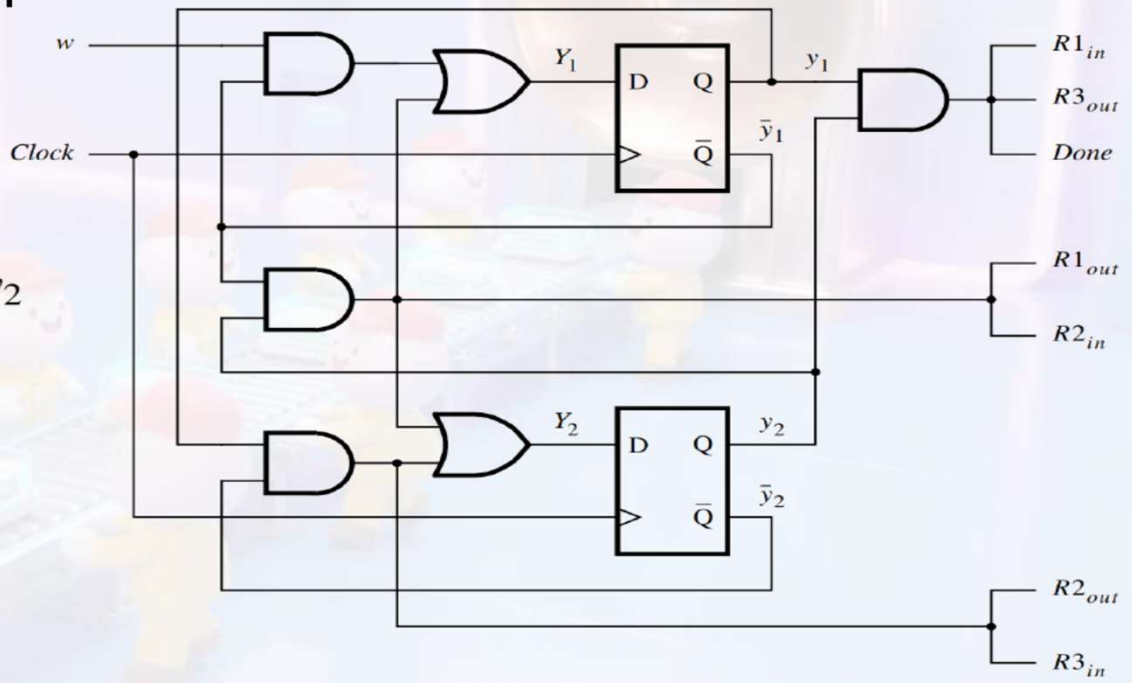
$$R2_{out} = R3_{in} = y_1 \bar{y}_2$$

同步时序电路

Register Swap

- Circuit implementation

$$Y_1 = w\bar{y}_1 + \bar{y}_1 y_2$$
$$Y_2 = y_1\bar{y}_2 + \bar{y}_1 y_2$$
$$R1_{out} = R2_{in} = \bar{y}_1 y_2$$
$$R1_{in} = R3_{out} = Done = y_1 y_2$$
$$R2_{out} = R3_{in} = y_1\bar{y}_2$$



同步时序电路

Register Swap

- An alternative state assignment

	Present state	Next state		Outputs						
		$w = 0$	$w = 1$							
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	$R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	0 0	0 0	0 1	0	0	0	0	0	0	0
B	0 1	1 1	1 1	0	0	1	0	0	1	0
C	1 1	1 0	1 0	1	0	0	1	0	0	0
D	1 0	0 0	0 0	0	1	0	0	1	0	1

$$Y_1 = w\bar{y}_1 + \bar{y}_1 y_2$$
$$Y_2 = y_1\bar{y}_2 + \bar{y}_1 y_2$$

simpler

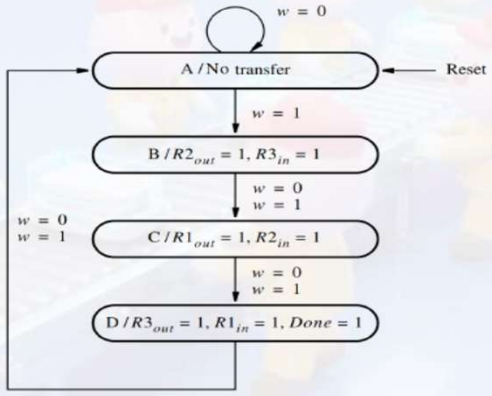
$$Y_1 = w\bar{y}_2 + y_1\bar{y}_2$$
$$Y_2 = y_1$$

同步时序电路

Verilog Code for Register Swap

```
module control (Clock, Resetn, w,
                R1in, R1out, R2in, R2out, R3in, R3out, Done);
    input Clock, Resetn, w;
    output R1in, R1out, R2in, R2out, R3in, R3out, Done;
    reg [2:1] y, Y;
    parameter [2:1] A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

    // Define the next state combinational circuit
    always @(w, y)
        case (y)
            A:  if (w) Y = B;
                else Y = A;
            B:  Y = C;
            C:  Y = D;
            D:  Y = A;
        endcase
```




```
    // Define the sequential block
    always @(negedge Resetn, posedge Clock)
        if (Resetn == 0) y <= A;
        else y <= Y;

    // Define outputs
    assign R2out = (y == B);
    assign R3in = (y == B);
    assign R1out = (y == C);
    assign R2in = (y == C);
    assign R3out = (y == D);
    assign R1in = (y == D);
    assign Done = (y == D);

endmodule
```

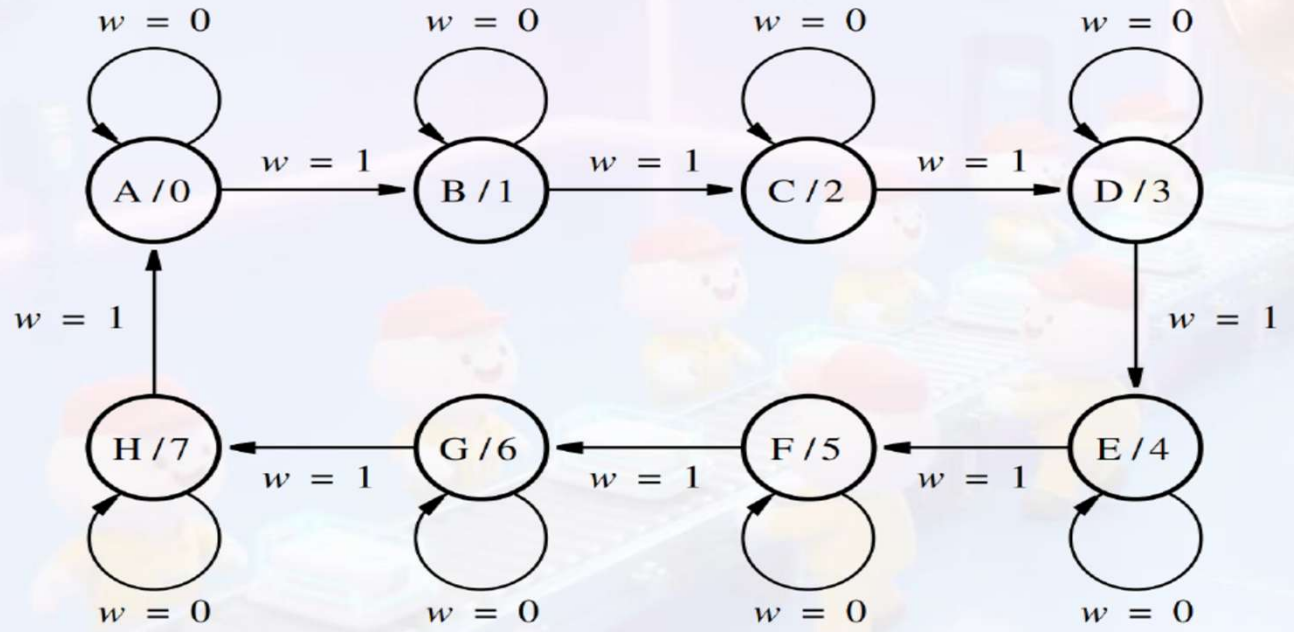
同步时序电路

- Circuit specification:
  - The counting sequence is 0, 1, 2, . . . , 6, 7, 0, 1, . . .
  - There exists an input signal w
  - During each clock cycle:
    if $w = 0$, count remains the same
    if $w = 1$, the count is incremented

同步时序电路

Counter as FSM

- State diagram

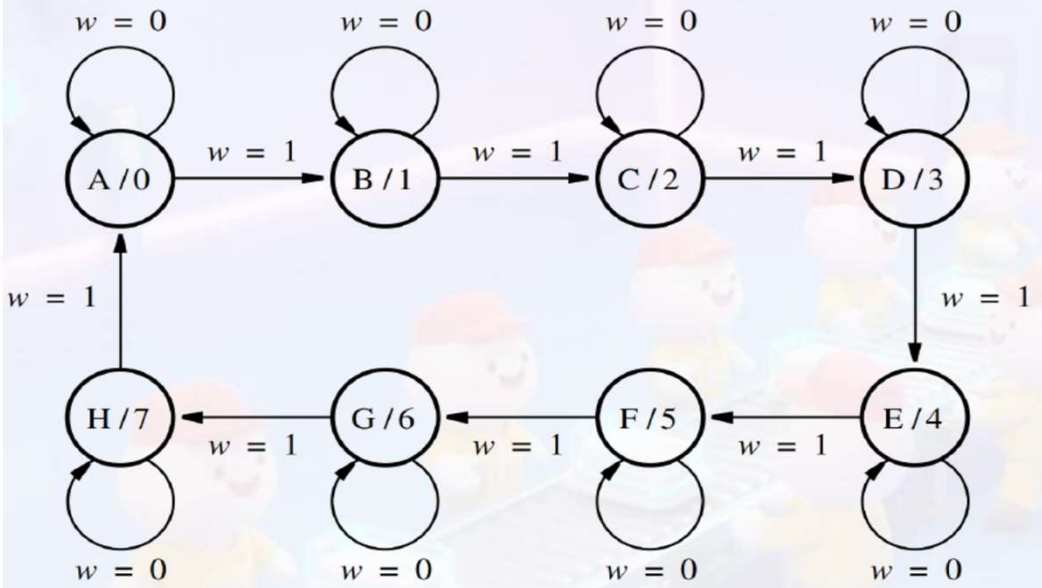


A: count = 0
B: count = 1
C: count = 2
D: count = 3
E: count = 4
F: count = 5
G: count = 6
H: count = 7

同步时序电路

Counter as FSM

- State table



Present state	Next state		Output
	$w = 0$	$w = 1$	
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7

同步时序电路

Counter as FSM

- Assigned state table

Present state	Next state $w = 0$	Next state $w = 1$	Output
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7



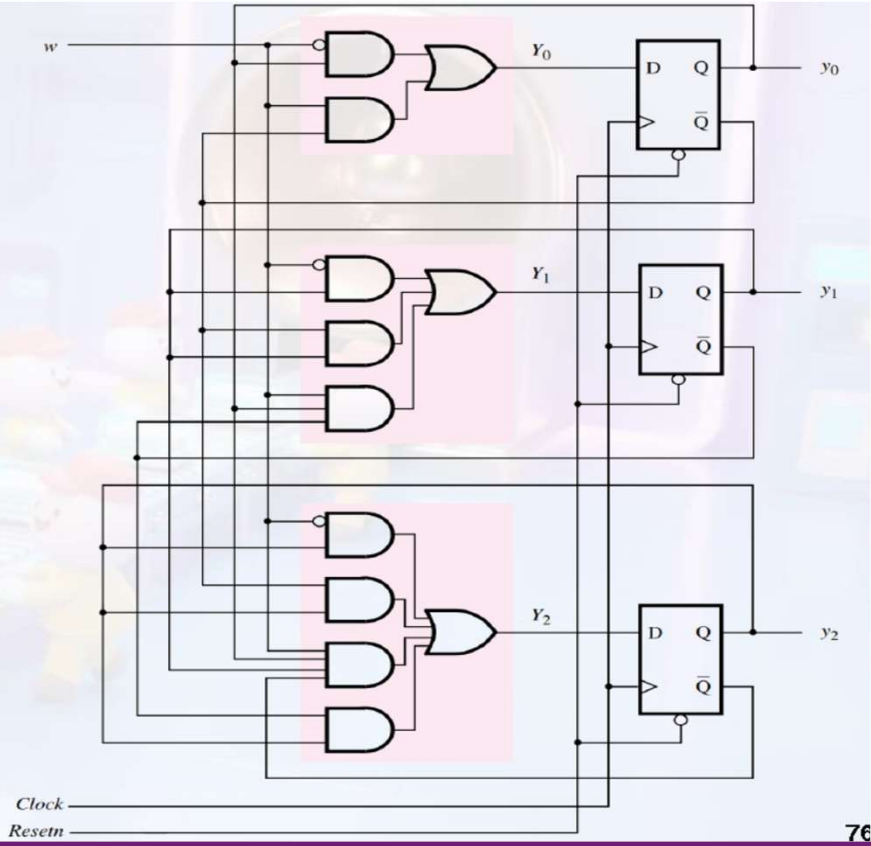
	Present state $y_2y_1y_0$	Next state $w = 0$ $Y_2Y_1Y_0$	Next state $w = 1$ $Y_2Y_1Y_0$	Count $z_2z_1z_0$
A	000	000	001	000
B	001	001	010	001
C	010	010	011	010
D	011	011	100	011
E	100	100	101	100
F	101	101	110	101
G	110	110	111	110
H	111	111	000	111

同步时序电路

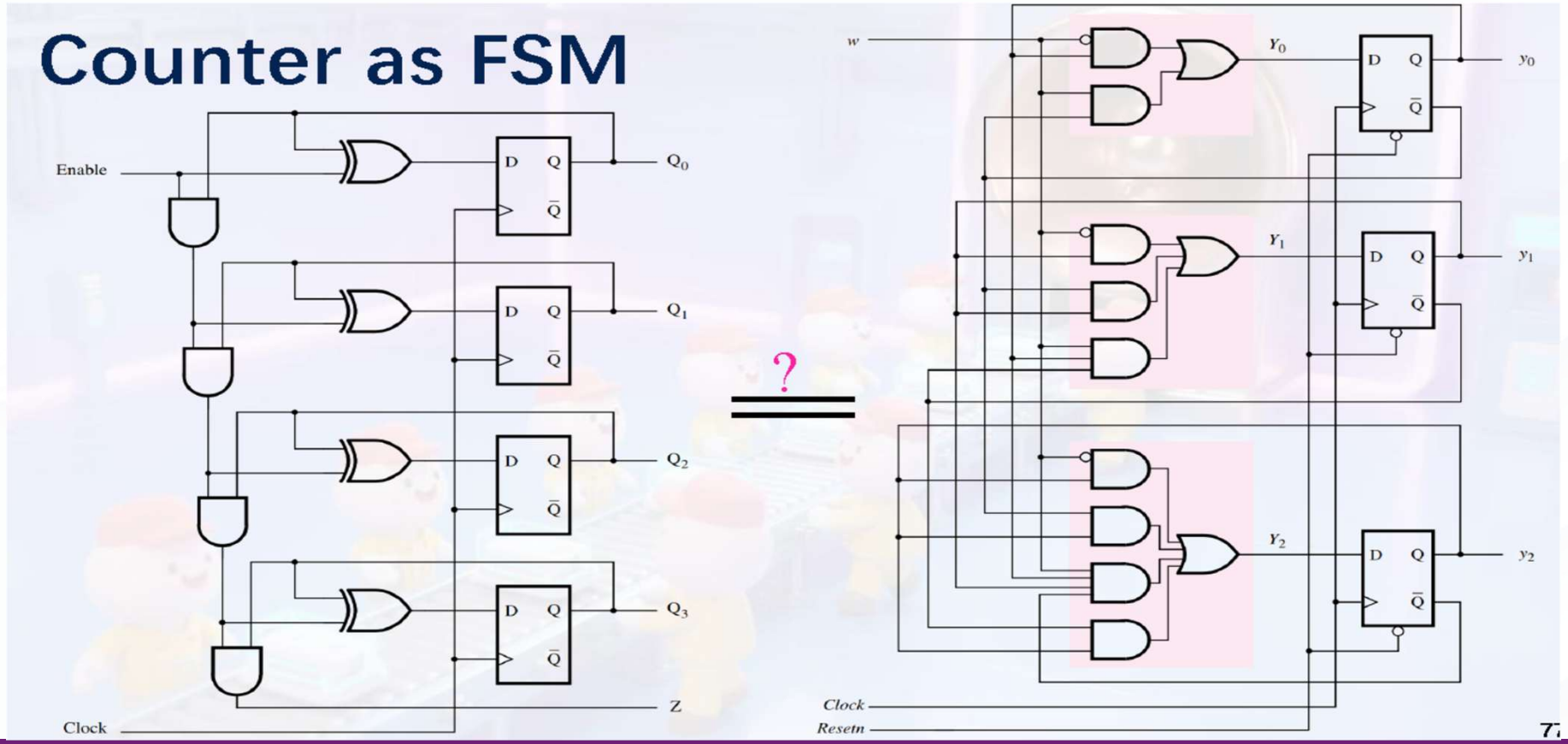
Counter as FSM

- Expression and circuit
  - Use D flip-flop

$$D_0 = Y_0 = \overline{w}y_0 + w\overline{y}_0$$
$$D_1 = Y_1 = \overline{w}y_1 + y_1\overline{y}_0 + wy_0\overline{y}_1$$
$$D_2 = Y_2 = \overline{w}y_2 + \overline{y}_0y_2 + \overline{y}_1y_2 + wy_0y_1\overline{y}_2$$



同步时序电路


Counter as FSM
Enable
D
Q
$\bar{Q}$
Q_0
Q_1
Q_2
Q_3
Z
Clock
?
w
Y_0
Y_1
Y_2
y_0
y_1
y_2
Clock
Resetn

同步时序电路

Counter as FSM

- Expression conversion

$$D_0 = Y_0 = \overline{w}y_0 + w\overline{y}_0$$
$$D_1 = Y_1 = \overline{w}y_1 + y_1\overline{y}_0 + wy_0\overline{y}_1$$
$$D_2 = Y_2 = \overline{w}y_2 + \overline{y}_0y_2 + \overline{y}_1y_2 + wy_0y_1\overline{y}_2$$

$$\begin{aligned} D_0 &= \overline{w}y_0 + w\overline{y}_0 \\ &= w \oplus y_0 \end{aligned}$$

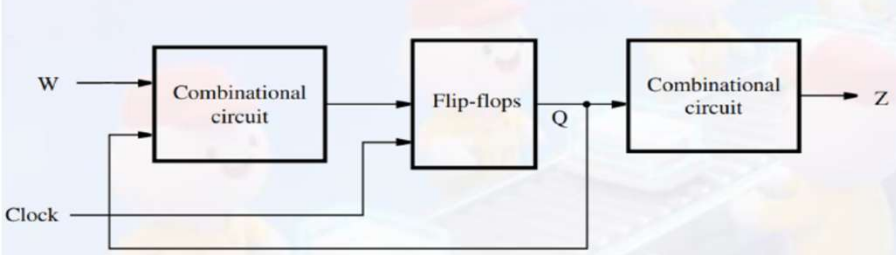
$$\begin{aligned} D_1 &= \overline{w}y_1 + y_1\overline{y}_0 + wy_0\overline{y}_1 \\ &= (\overline{w} + \overline{y}_0)y_1 + wy_0\overline{y}_1 \\ &= \overline{w\overline{y}_0}y_1 + wy_0\overline{y}_1 \\ &= wy_0 \oplus y_1 \end{aligned}$$

$$\begin{aligned} D_2 &= \overline{w}y_2 + \overline{y}_0y_2 + \overline{y}_1y_2 + wy_0y_1\overline{y}_2 \\ &= (\overline{w} + \overline{y}_0 + \overline{y}_1)y_2 + wy_0y_1\overline{y}_2 \\ &= \overline{wy_0\overline{y}_1}y_2 + wy_0y_1\overline{y}_2 \\ &= wy_0y_1 \oplus y_2 \end{aligned}$$

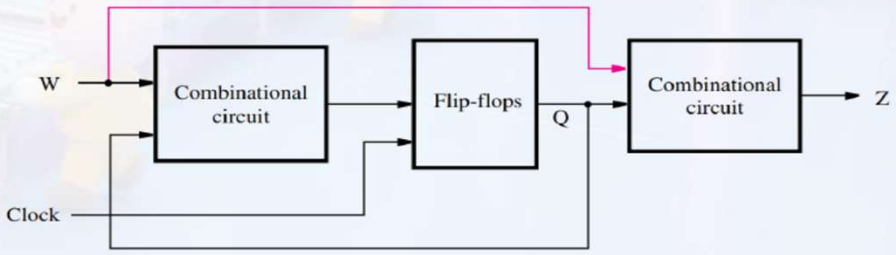
同步时序电路

Moore vs. Mealy

- Moore-type FSM:
  - outputs depend only on the state of the circuit
  - in honor of Edward Moore
- Mealy-type FSM:
  - outputs depend on both the state and the **inputs**
  - in honor of George Mealy



Moore-type



Mealy-type

同步时序电路

Moore vs. Mealy

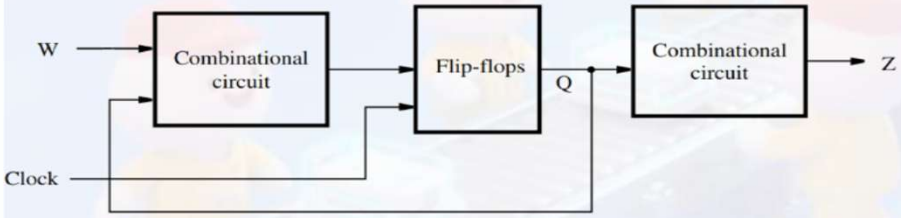
- What kind are the two examples?

Smiling snail

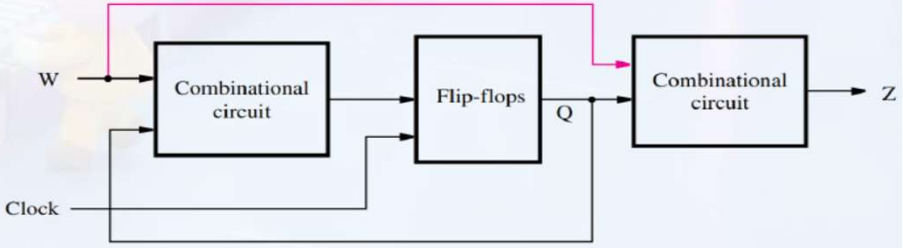
Present state	Next state $w = 0$	Next state $w = 1$	Output z
A	A	B	0
B	A	C	0
C	A	C	1

Register swap

Present state	Next state $w = 0$	Next state $w = 1$	Outputs $R1_{out}$	$R1_{in}$	$R2_{out}$	$R2_{in}$	$R3_{out}$	$R3_{in}$	$Done$
A	A	B	0	0	0	0	0	0	0
B	C	C	0	0	1	0	0	1	0
C	D	D	1	0	0	1	0	0	0
D	A	A	0	1	0	0	1	0	1



Moore type

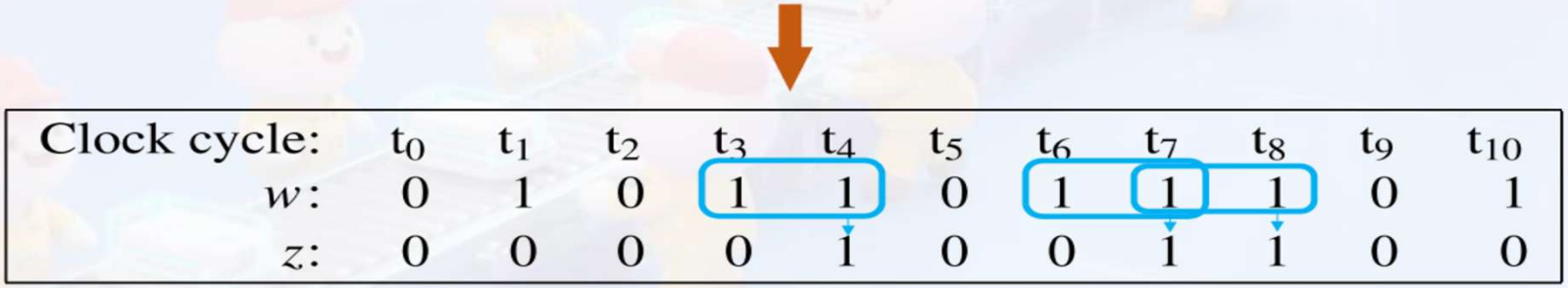


Mealy type

Mealy FSM for "Smiling Snail"

- Modify one specification:
  The output $z = 1$ in the same clock cycle when the second occurrence of $w = 1$ is detected

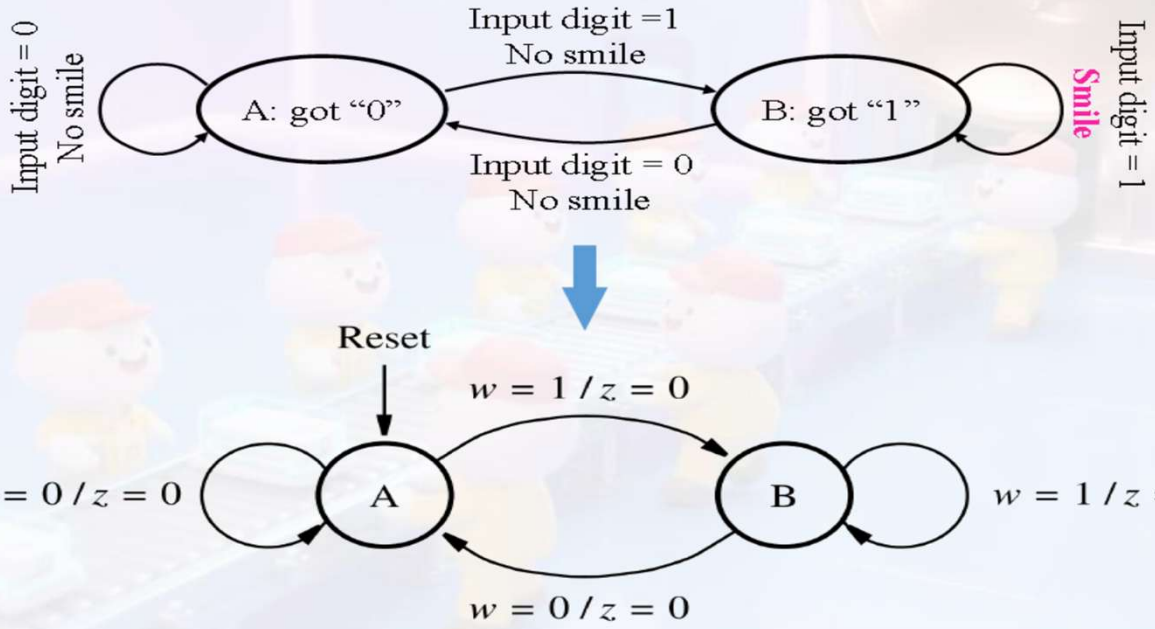
Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0



同步时序电路

Mealy FSM for “Smiling Snail”

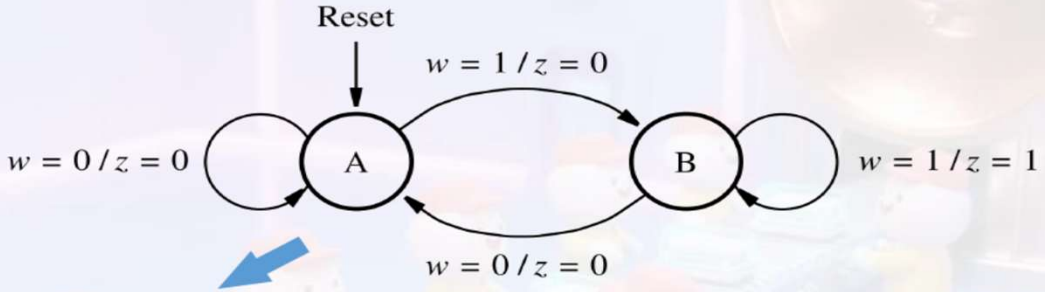
- Redesign the circuit: state diagram



同步时序电路

Mealy FSM for “Smiling Snail”

- Redesign: state table & state assignment



Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Present state	Next state		Output	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

同步时序电路

Mealy FSM for "Smiling Snail"

- Redesign: next-state and output expression

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

w	y	Y
0	0	0
0	1	0
1	0	1
1	1	1

$Y = w$

w	y	z
0	0	0
0	1	0
1	0	0
1	1	1

$z = wy$

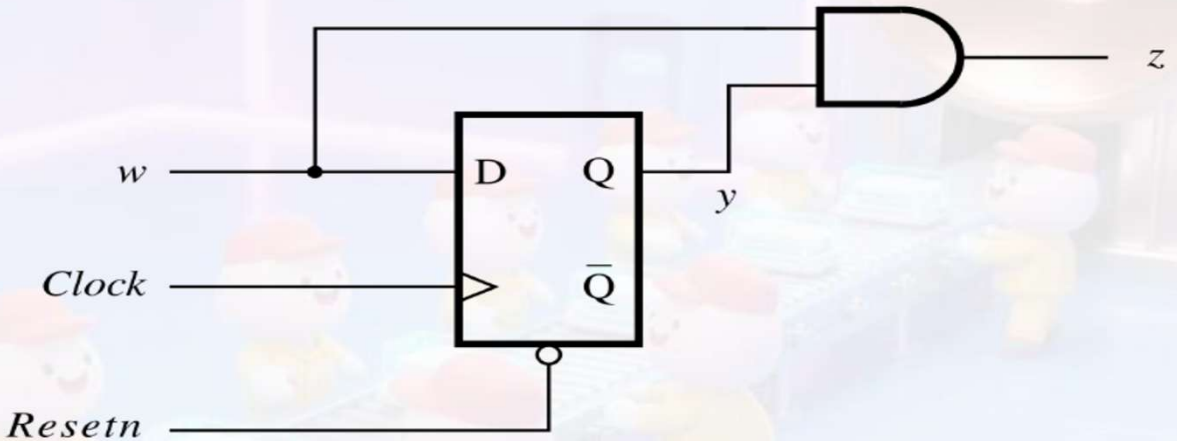
同步时序电路

Mealy FSM for “Smiling Snail”

- Redesign: circuit implementation

$$Y = w$$

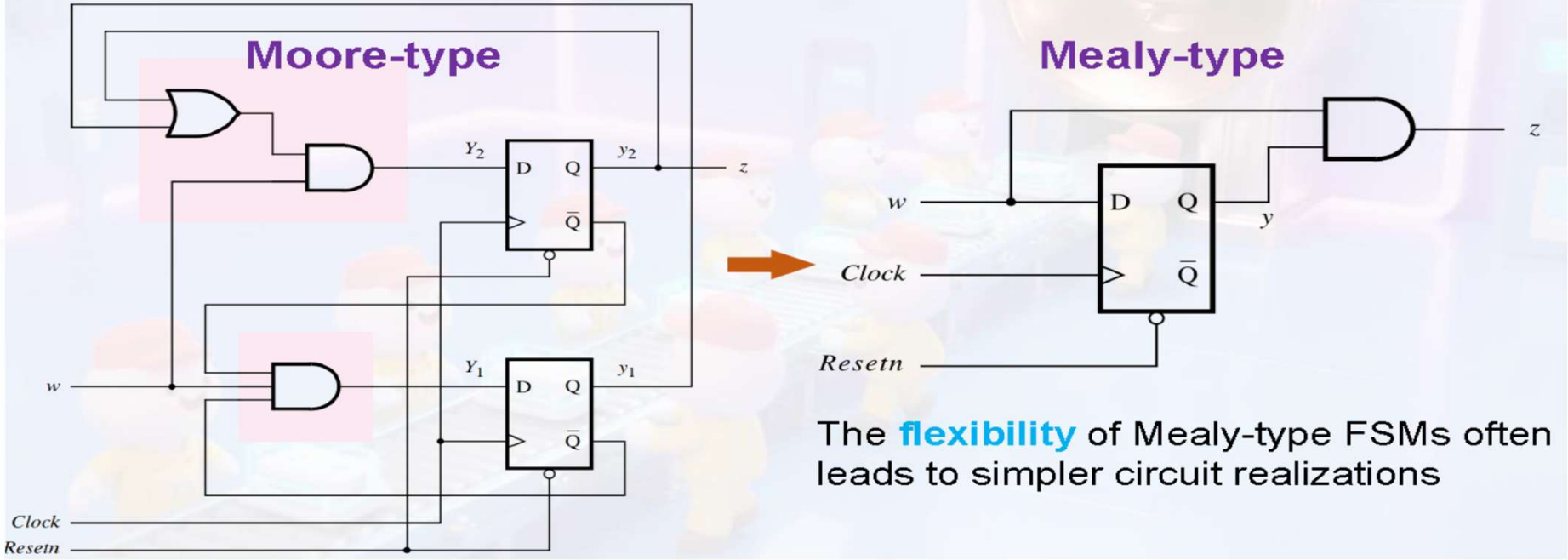
$$z = wy$$



同步时序电路

Mealy FSM for “Smiling Snail”

- Comparison between Moore and Mealy



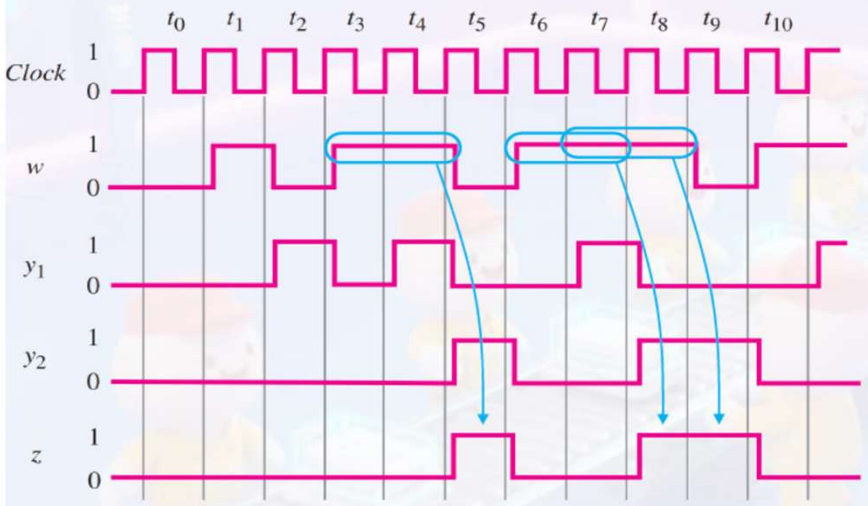
The **flexibility** of Mealy-type FSMs often leads to simpler circuit realizations

同步时序电路

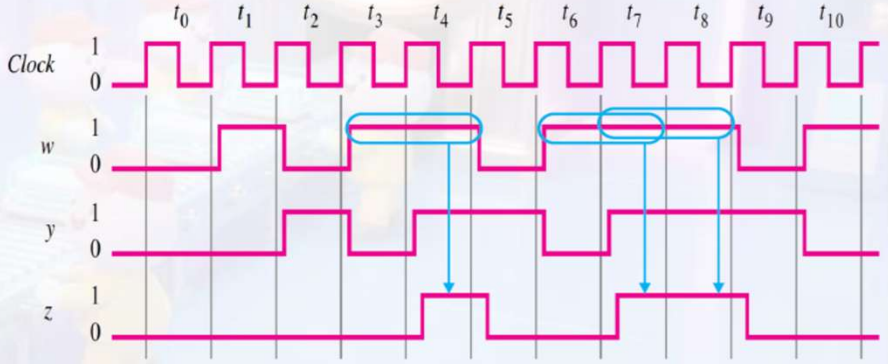
Mealy FSM for “Smiling Snail”

- Comparison between Moore and Mealy

Moore-type



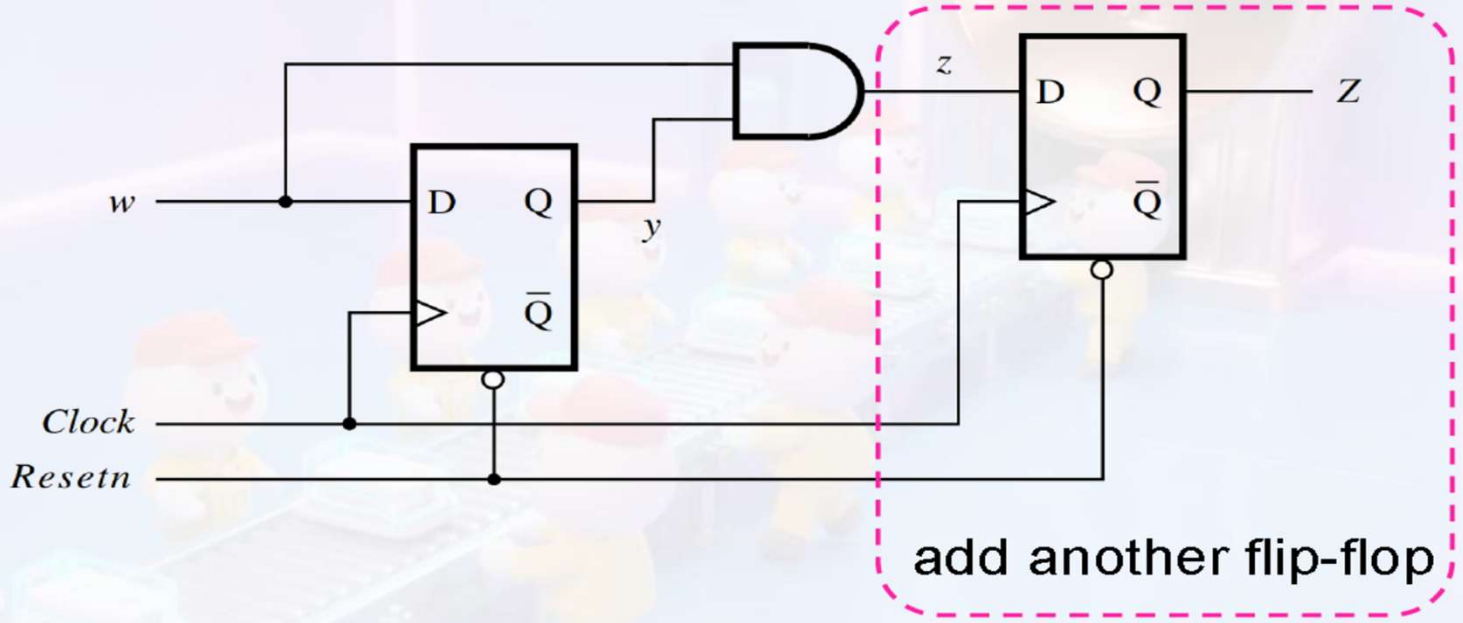
Mealy-type



同步时序电路

Mealy FSM for “Smiling Snail”

- Convert Mealy to Moore



同步时序电路

Verilog code for Mealy-type FSM

```
module mealy (Clock, Resetn, w, z);
    input Clock, Resetn, w;
    output reg z;
    reg y, Y;
    parameter A = 1'b0, B = 1'b1;

    // Define the next state and output combinational circuits
    always @(w, y)
        case (y)
            A: if (w) begin
                    z = 0;
                    Y = B;
                end
                else begin
                    z = 0;
                    Y = A;
                end
            B: if (w) begin
                    z = 1;
                    Y = B;
                end
                else begin
                    z = 0;
                    Y = A;
                end
        endcase

    // Define the sequential block
    always @(negedge Resetn, posedge Clock)
        if (Resetn == 0) y <= A;
        else y <= Y;

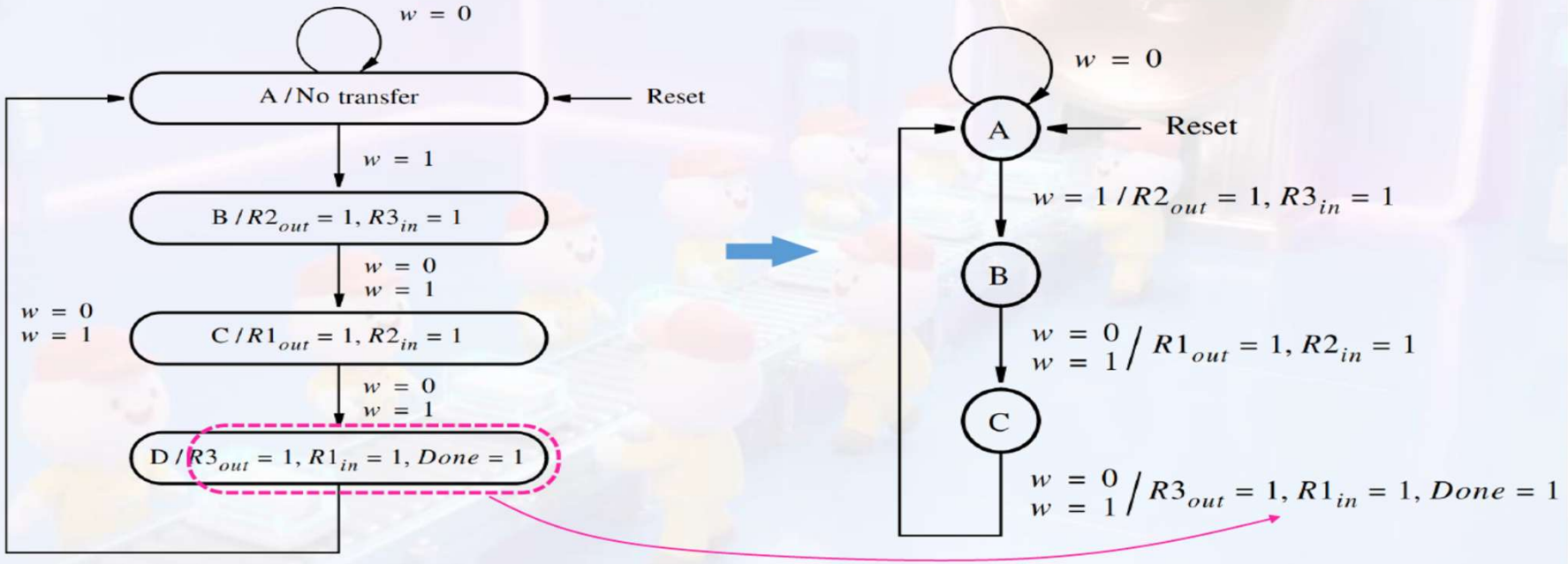
endmodule
```

output is specified under every input-state combination

同步时序电路

Mealy FSM for "Register Swap"

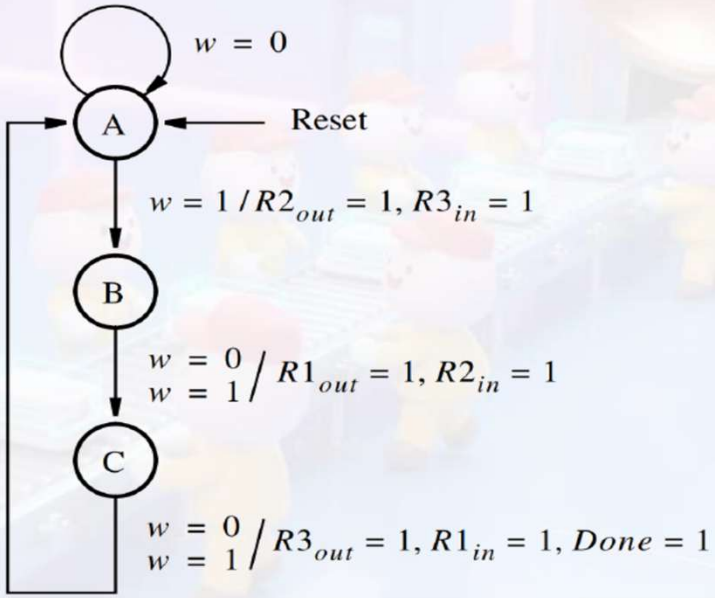
- Three states left, one state reduced



同步时序电路

Practice:

Problem: Derive the state table, expressions of next-state and output, and circuit implementation based on the mealy-type state diagram below.



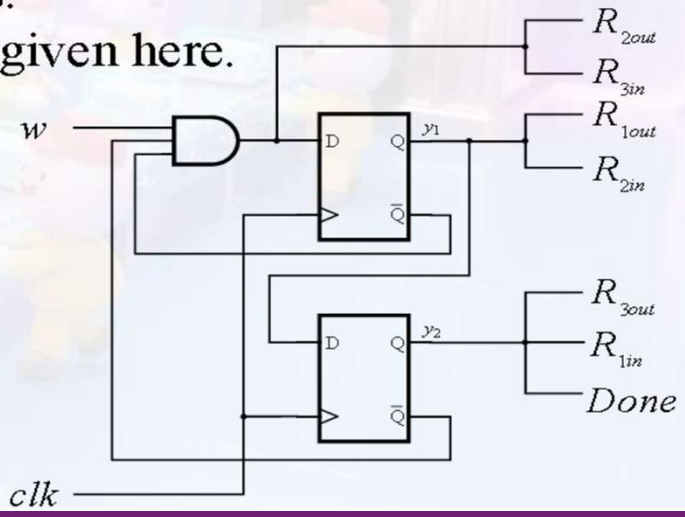
同步时序电路

Practice:

Problem: Derive the state table, expressions of next-state and output, and circuit implementation based on the mealy-type state diagram below.

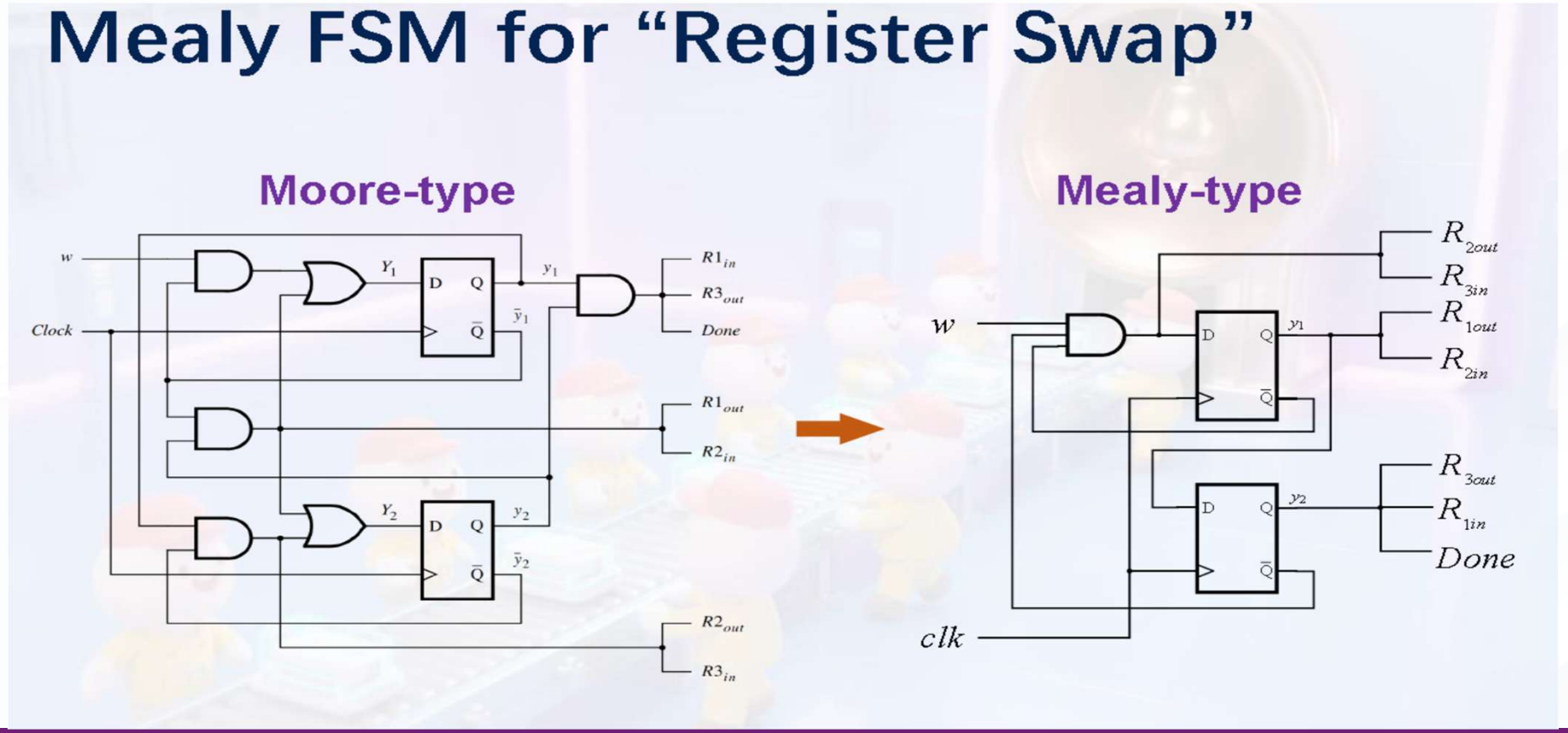
Solution: (1) Try derivation of "the state table, expressions of next-state and output" by yourselves.

(2) Circuit implementation is given here.



同步时序电路

Mealy FSM for “Register Swap”



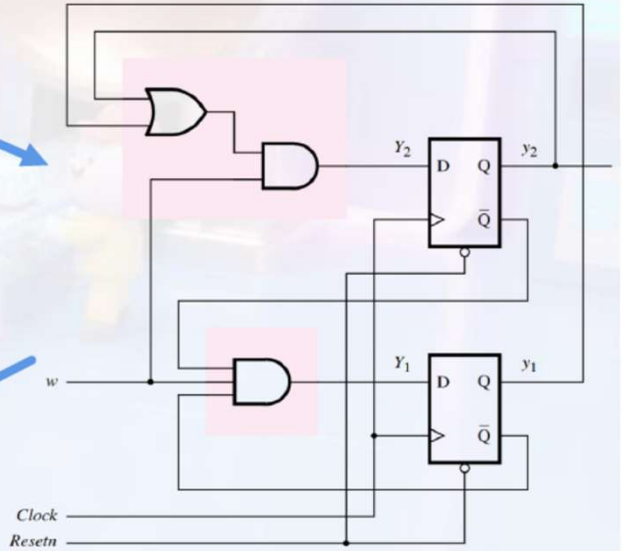
同步时序电路

Analysis of Synchronous Sequential Circuits

- Analyze the behavior of an existing circuit

Synthesize 综合

Circuit specification
- w : the input digit
- z : output action (smile "1", no smile "0")
- all changes occur on the positive edge of the clock signal
- $z = 1$, if "$w = 1$" happens during two preceding clock cycles
  $z = 0$, otherwise

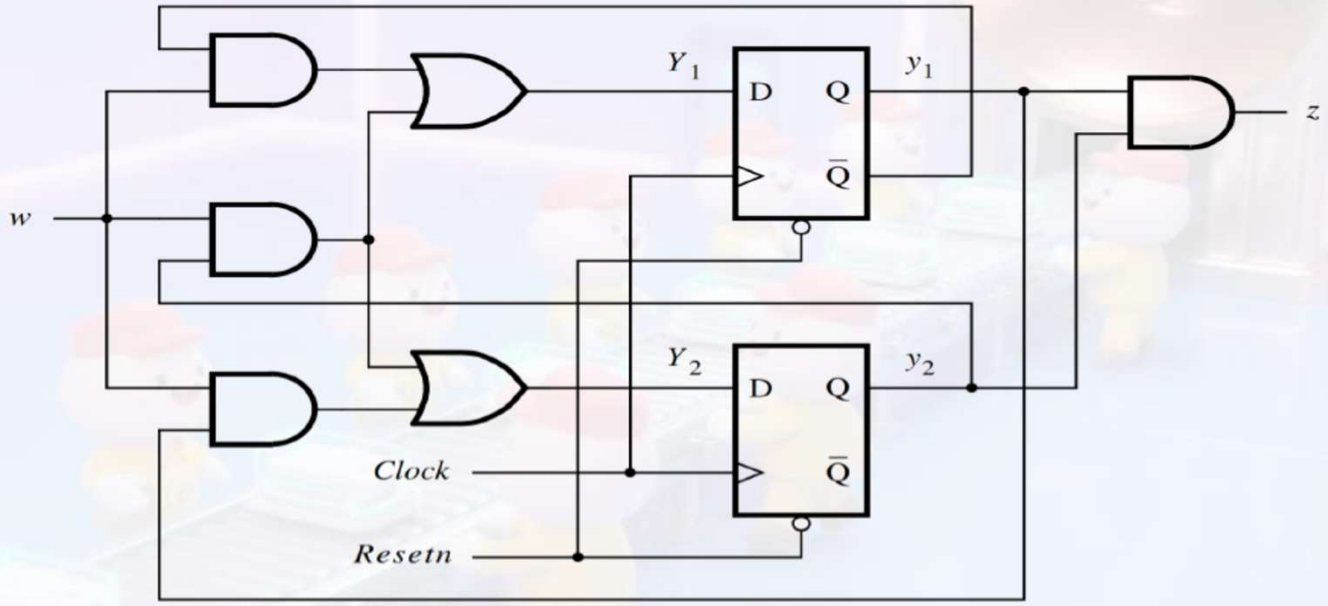


Analyze 分析

同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.



同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.

Answer:

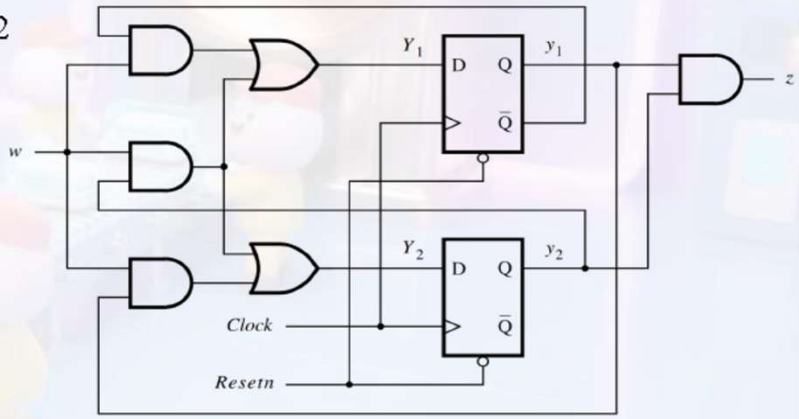
step1: present-state variables: y_1 and y_2
next-state variables: Y_1 and Y_2

step2: the next-state and output expressions are

$$Y_1 = w\bar{y}_1 + wy_2$$

$$Y_2 = wy_1 + wy_2$$

$$z = y_1 y_2$$



同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.

Answer:

step3: assume an initial state of the flip-flops such as $y_1 = y_2 = 0$, then derive the state-assigned table (状态分配表) from the expressions

$$Y_1 = w\bar{y}_1 + wy_2$$
$$Y_2 = wy_1 + wy_2$$
$$z = y_1y_2$$

Present state y_2y_1	Next state $w = 0$ Y_2Y_1	Next state $w = 1$ Y_2Y_1	Output z
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1

同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.

Answer:

step4: derive the state table (状态表)

Present state y_2y_1	Next state $w = 0$ Y_2Y_1	Next state $w = 1$ Y_2Y_1	Output z
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1



Present state	Next state $w = 0$	Next state $w = 1$	Output z
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.

Answer:

step5: the output $z = 1$ whenever three consecutive 1s occur on the input w. Therefore, the FSM acts as a sequence detector (序列检测).

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

同步时序电路

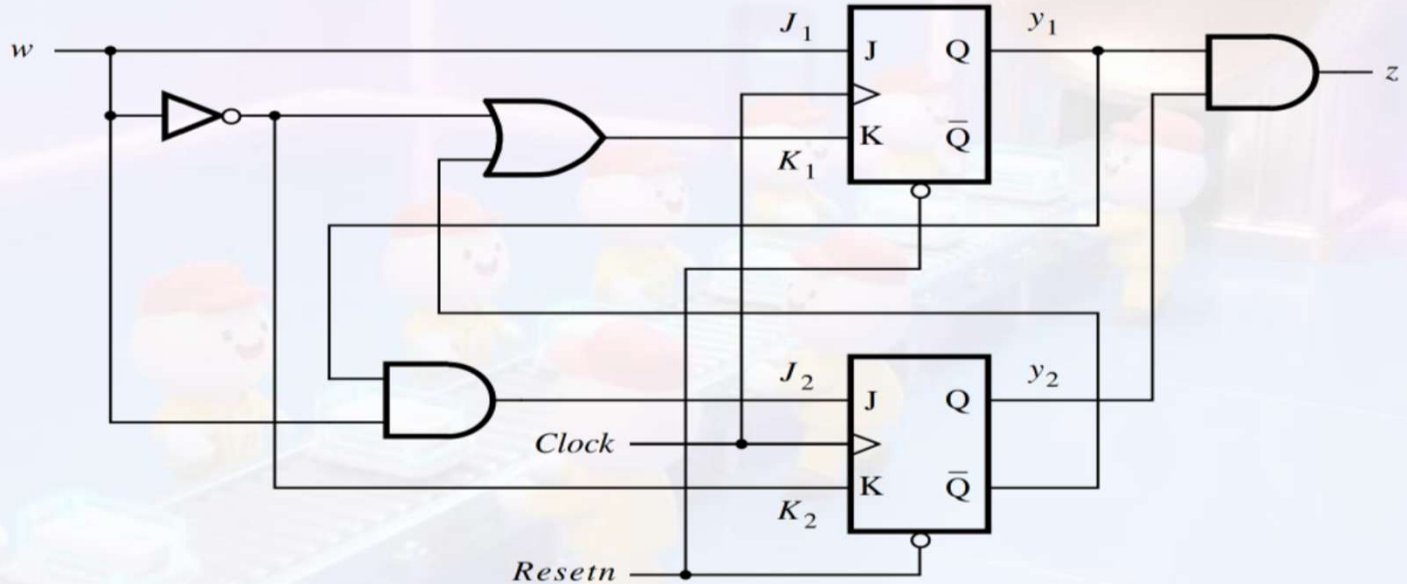
Steps of Analysis

1. Determine the state variables
2. Derive the logic expressions of next-state and output
3. Derive the state-assigned table
4. Derive the state table from the state-assigned table
5. Analyze the function according to the state table

同步时序电路

Practice:

Problem: Try to analyze the function of the following circuit.



同步时序电路

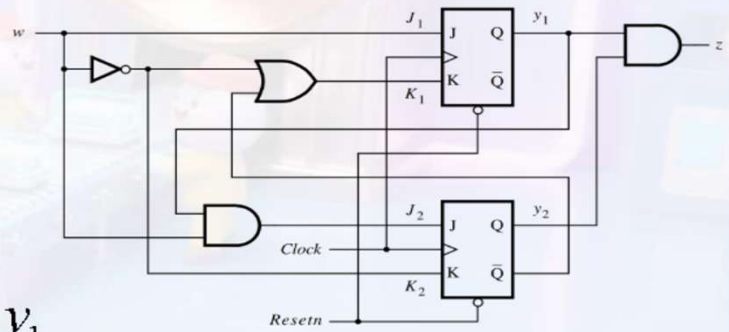
Practice:

Problem: Try to analyze the function of the following circuit.

Answer:

step1: present-state variables: y_1 and y_2
next-state variables: Y_1 and Y_2

step2: the J, K next state and output expressions are



$$\begin{cases} J_1 = w \\ K_1 = \overline{w} + \overline{y}_2 \\ J_2 = wy_1 \\ K_2 = \overline{w} \end{cases} \qquad \begin{cases} Y_1 = J_1\overline{y}_1 + \overline{K}_1 y_1 \\ Y_2 = J_2\overline{y}_2 + \overline{K}_2 y_2 \\ z = y_1 y_2 \end{cases}$$

同步时序电路

Practice:

Answer:

step3: assume an initial state of the flip-flops such as $y_1 = y_2 = 0$, then derive the state-assigned table (状态分配表) from the expressions

$$\begin{cases} J_1 = w \\ K_1 = \overline{w} + \overline{y}_2 \\ J_2 = w y_1 \\ K_2 = \overline{w} \end{cases}$$

$$\begin{cases} Y_1 = J_1 \overline{y}_1 + \overline{K}_1 y_1 \\ Y_2 = J_2 \overline{y}_2 + \overline{K}_2 y_2 \\ z = y_1 y_2 \end{cases}$$

Present state y_2y_1	Flip-flop inputs & next state						Output z
	$w = 0$			$w = 1$			
	J_2K_2	J_1K_1	Y_2Y_1	J_2K_2	J_1K_1	Y_2Y_1	
00	01	01	00	00	11	01	0
01	01	01	00	10	11	10	0
10	01	01	00	00	10	11	0
11	01	01	00	10	10	11	1

Same as last example!